



MODELISATION DES SYSTEMES D'INFORMATION : APPROCHES OBJET AVEC UML

Chapitre 2 : Le Langage de Modélisation UML

Introduction

L'informatisation consiste à transformer les systèmes d'information existants par :

- une substitution des moyens (automatisation des tâches),
- une diffusion rapide de l'information,
- la qualité de présentation de l'information.

Toutefois, la réalisation de l'efficacité de l'informatisation au sein d'une organisation doit passer par la conception d'un modèle. Un modèle est une représentation abstraite d'une entité du monde réel en vue de le décrire ou de l'interpréter. Il est plus aisé de se référer à un modèle qu'à l'entité d'origine, car le modèle simplifie la gestion de la complexité en offrant des points de vue et des niveaux d'abstractions plus ou moins détaillés selon les besoins. L'abstraction, dans ce contexte, signifie l'examen sélectif de certains aspects du problème.

1. Pourquoi Modéliser ?

Modéliser un système avant sa réalisation permet de mieux comprendre le fonctionnement du système et de capturer tous les besoins fonctionnels. C'est également un bon moyen de maîtriser sa complexité et d'assurer sa cohérence. Le but de la modélisation en informatique est de pouvoir ressortir un logiciel de qualité c'est-à-dire qui respecte les critères suivants :

- **Validité** : aptitude d'un produit logiciel à remplir exactement ses fonctions, définies par le cahier des charges et les spécifications ou exigences des utilisateurs.
- **Robustesse ou fiabilité** : aptitude d'un produit logiciel à fonctionner quelque soit les conditions d'exploitation (montée en charge, panne d'un composant).
- **Maintenabilité/Extensibilité** : facilité à apporter des corrections ou à ajouter des fonctions au logiciel.



- **Réutilisabilité** : aptitude d'un logiciel à être réutilisé, en tout ou en partie, dans de nouvelles applications.
- **Compatibilité** : facilité avec laquelle un logiciel peut être combiné avec d'autres logiciels.
- **Portabilité** : aptitude d'un logiciel de fonctionner dans un environnement matériel ou logiciel différent de son environnement initial.
- **Intégrité** : aptitude d'un logiciel à protéger son code et ses données contre des accès non autorisés.
- **Facilité d'emploi** : facilité d'apprentissage, d'utilisation, de préparation des données, d'interprétation des erreurs et de rattrapage en cas d'erreur d'utilisation.

En résumé, on modélise pour décrire de manière visuelle et graphique les besoins et, les solutions fonctionnelles et techniques de votre projet logiciel.

2. Le cycle de vie d'un logiciel

Afin de résoudre les problèmes qui ont causé la **crise du logiciel** dans les années 1970 principalement dus à :

- l'augmentation des coûts ;
- les difficultés de maintenance et d'évolution ;
- la non-fiabilité ;
- le non-respect des spécifications ;
- le non-respect des délais.

Le génie logiciel propose un ensemble d'étapes qui, respectées permettent d'aboutir à un logiciel de qualité appelées cycle de vie d'un logiciel (terme qui désigne toutes les étapes du développement d'un logiciel de sa conception à sa disparition).

Les étapes du cycle de vie d'un logiciel sont les suivantes :

- l'étude de faisabilité,



- l'élaboration des spécifications ou expression des besoins,
- l'analyse,
- la conception,
- le développement,
- codage,
- test,
- la maintenance.

Pratique : Modèles de cycles de vie d'un logiciel

3. Généralités sur UML

De la fin des années 1980 aux années 1990, de nombreuses méthodes et langages pour la représentation de la POO ont été développés et mis en œuvre. Il en est résulté une variété de méthodes très dissemblables. Pour unifier ces langages, les trois développeurs James Rumbaugh, Grady Booch et Ivar Jacobson ont décidé de fusionner plusieurs langages existants (la méthode Booch, l'Object modeling technique ou OMT, l'object-oriented software engineering method ou OOSE) en un langage commun et standardisé : UML.

UML pour Unified Modeling Language, est un langage de modélisation graphique à base de schémas appelés diagrammes conçu pour fournir une norme de visualisation lors de la conception d'un logiciel.

C'est en 1997 que l'OMG (Object Management Group qui est un consortium dont le but est de promouvoir et de standardiser l'orienter objet) l'UML 1.0. Non satisfaits, les développeurs ont mis en place un groupe de travail pour améliorer le langage sur plusieurs versions. Pour cette raison, une révision majeure a été effectuée on est quitté à cet effet de la 1.0 à la 1.5 qui comporte 9 diagrammes.

À ce jour la dernière version de la spécification validée par l'OMG en 2017 est UML 2.5.1 (qui apporte des innovations radicales et étendant largement le champ d'application d'UML.

) et qui comporte 14 diagrammes.



N.B :

UML ne préconise aucune démarche, ce n'est donc pas une méthode. Chacun est libre d'utiliser les types de diagramme qu'il souhaite, dans l'ordre qu'il veut. Il suffit que les diagrammes réalisés soient cohérents entre eux, avant de passer à la réalisation du logiciel.

4. L'approche objet

UML est un langage orienté objet c'est-à-dire qu'il considère le logiciel comme une collection d'objets dissociés, identifiés et possédant des caractéristiques (les attributs : variables stockant les informations sur l'objet, et les méthodes : ensemble d'actions ou opérations réalisables par l'objet). La particularité de l'orienté objet est qu'elle rapproche les données et les traitements.

Cette approche est une démarche qui s'organise autour de 3 principales fonctions:

- Elle est Itérative et incrémentale : qui consiste à faire des allers-retours entre le plan initial et les modifications apportées par les acteurs du projet.
- Guidée par les besoins du client et des utilisateurs : ces besoins représentent les exigences du client qui devront être implémentés et qui serviront de fil conducteur tout au long du cycle de développement.
- Centrée sur l'architecture du logiciel : L'architecture logicielle décrit les choix stratégiques qui déterminent en grande partie les qualités du logiciel (adaptabilité, performances, fiabilité...). On peut citer l'architecture client serveur, en couche ou en niveaux. Il s'agit ici de se poser des questions de base à savoir: Quels seront les différents composants logiciels à utiliser (base de données, librairies, interfaces, etc.) ? Sur quel matériel chacun des composants sera installé ?

Quelques concepts de l'Orienté objet

Certaines notions importantes sont spécifiques à l'approche objet et participent à la mise en place d'un logiciel de qualité :



- La classe : est un type de données abstrait qui précise des caractéristiques (attributs et méthodes) communes à toute une famille d'objets et qui permet de créer des objets possédant ces caractéristiques. En d'autres termes une classe déclare des propriétés communes à un ensemble d'objets, c'est un moule à partir duquel il est possible de créer des objets. Ex : la classe Animal, avec pour attributs nom, espece, sexe, type.... Avec pour méthodes : se_deplacer(), crier()..
- L'encapsulation : Elle consiste à masquer les détails d'implémentation d'un objet, en définissant une interface. Cela signifie que l'utilisateur d'une classe n'a pas forcément à savoir de quelle façon sont structurées les données. L'interface est la vue externe d'un objet, elle définit les services accessibles (offerts) aux utilisateurs de l'objet. Elle garantit l'intégrité des données, car elle permet d'interdire, ou de restreindre, l'accès direct aux attributs des objets, ce qui permet de définir les niveaux de visibilité (droit d'accès des données) :
 - **public**: les fonctions de toutes les classes peuvent accéder aux données ou aux méthodes d'une classe définie avec le niveau de visibilité *public*. Il s'agit du plus bas niveau de protection des données.
 - **Protected** : l'accès aux données est réservé aux fonctions des classes héritières, c'est-à-dire par les fonctions membres de la classe ainsi que des classes dérivées.
 - **Private** : l'accès aux données est limité aux méthodes de la classe elle-même. Il s'agit du niveau de protection des données le plus élevé.
- L'héritage est un mécanisme de transmission des caractéristiques d'une classe (ses attributs et méthodes) vers une sous-classe. Une classe peut être spécialisée en d'autres classes, afin d'y ajouter des caractéristiques spécifiques ou d'en adapter certaines. Plusieurs classes peuvent être généralisées en une classe qui les factorise, afin de regrouper les caractéristiques communes d'un ensemble de classes. Ainsi, la spécialisation et la généralisation permettent de construire des hiérarchies de classes. L'héritage peut être simple (java,php) ou multiple(c++,python). L'héritage évite la duplication et encourage la réutilisation.



- Le polymorphisme représente la faculté d'une méthode à pouvoir s'appliquer à des objets de classes différentes. Le polymorphisme augmente la généricité, et donc la qualité, du code.

UML se décompose en plusieurs parties :

- Les *vues* : ce sont les observables du système. Elles décrivent le système d'un point de vue donné c'est à dire organisationnel, dynamique, temporel, architectural, géographique, logique, etc. En combinant toutes ces vues, il est possible de définir (ou retrouver) le système complet.
- Les *diagrammes* : ce sont des ensembles d'éléments graphiques. Ils décrivent le contenu des vues, qui sont des notions abstraites. Ils peuvent faire partie de plusieurs vues.

5. Les vues

Une façon de mettre en œuvre UML est de considérer différentes *vues* qui peuvent se superposer pour collaborer à la définition du système :

- *Vue des cas d'utilisation*: c'est la description du modèle vu par les acteurs du système. Elle correspond aux besoins attendus par chaque acteur. (diagrammes de paquetage, cas d'utilisation)
- *Vue logique*: c'est la définition du système vu de l'intérieur. Elle explique comment peuvent être satisfaits les besoins des acteurs. Elle décrit le système en termes de classes, d'objets, de communications (elle identifie les éléments du domaine, les relations et les interactions entre ces éléments). (diagramme de classe, d'objets)
- *Vue des composants*: met en évidence les différentes parties qui composeront le futur système (fichiers sources, bibliothèques, bases de données, exécutables, etc.). Cette vue comprend deux diagrammes.(structure composite, composant)
- *Vue des processus*: elle décrit, les interactions entre les processus, la synchronisation et la communication des activités parallèles.(séquence, activité, collaboration, état-transition, global d'interaction, temps)



- *Vue de déploiement*: cette vue décrit l'architecture physique de chaque élément du système et la répartition des parties du logiciel sur ces éléments. (déploiement)

6. Les diagrammes

UML 2.5 comporte 14 diagrammes répartis en deux grands groupes :

- Les diagrammes comportementaux ou dynamiques: on y retrouve les diagrammes suivants :
 - Le diagramme des cas d'utilisation
 - Le Diagramme d'activité
 - Le Diagramme d'état-transition

Dans cette catégorie on retrouve les diagrammes comportementaux dits diagrammes d'interaction :

- Le diagramme de communication ou de collaboration (depuis UML 2.0)
 - Le diagramme de séquence
 - Le diagramme de temps (depuis UML 2.0)
 - Le diagramme global d'interaction (depuis UML 2.0)
- Les diagrammes structurels ou statiques : on y retrouve :
 - Le diagramme de paquetage
 - Le diagramme de classe
 - Le diagramme de structure composites (depuis UML 2.0)
 - Le diagramme d'objets
 - Le diagramme de déploiement
 - Le diagramme de composants
 - Le diagramme de profil (depuis UML 2.2)



6.1 Les diagrammes comportementaux ou dynamiques

Les **diagrammes de comportement**, pour leur part, ne portent pas sur les structures statiques, mais montrent plutôt sur le flux de processus prévu ou existant, comme il est attendu à l'utilisation du programme ou du logiciel. Ils décrivent le système en terme d'interactions sur deux plans particuliers:

- Système vu comme une boîte noire : Qui interagit avec le système, et dans quel but ? (Diagramme de cas d'utilisation); Comment interagit le système avec son environnement ? (Diagramme de séquence, activité)
- Système vu comme une boîte blanche : Comment interagissent les objets du système ? (Diagramme de séquence et/ou de communication); Décrire l'évolution du système dans le temps, Comment évoluent les états des objets ? (Diagrammes d'états-transitions)

6.1.1 Diagramme de cas d'utilisation

Un diagramme de cas d'utilisation permet de représenter les **fonctions d'un système du point de vue de l'utilisateur** (appelé « acteur » en UML). Les cas d'utilisation permettent de structurer les besoins des utilisateurs et les objectifs correspondants d'un système en définissant le contour ou le périmètre du système.

En effet, les cas d'utilisation identifient les utilisateurs du système (acteurs) et leurs interactions avec le système, permettant ainsi d'identifier les demandes ou les attentes vis-à-vis du système.

Éléments de modélisation des cas d'utilisation

- **Acteur** : désigne un rôle joué par toute entité qui interagit avec le système (celui qui utilise et celui qui fournit un service).

NB :

- Un acteur n'est pas nécessairement un être humain, le rôle peut être attribué à un système externe au système d'étude, un service, une société, une API...
- Une même personne physique peut donc être représentée par plusieurs acteurs en fonction des rôles qu'elle joue (ex le rôle admin et directeur qui peuvent être attribué à la même personne).
- Pour identifier les acteurs, il faut donc se concentrer sur les rôles joués par les entités extérieures au périmètre.



- Dans UML, il n'y a pas de notion d'acteur interne et externe. Par définition, un acteur est externe au périmètre de l'étude, qu'il appartienne ou pas à la société.

Il existe 2 principales catégories d'acteurs :

- les acteurs principaux : les personnes qui utilisent les fonctions principales du système
- les acteurs secondaires : entités qui offrent des services au système

Formalisme :

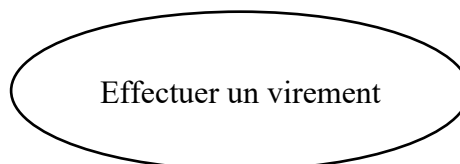
Il est possible de représenter un acteur sous forme d'un bonhomme ou sous forme d'un classeur :



- **Le cas d'utilisation :** Le cas d'utilisation (ou use case) correspond à un objectif du système, motivé par un besoin d'un ou plusieurs acteurs. L'ensemble des use cases décrit les objectifs (le but) du système. Il fournit une vue de haut niveau de comportement observable à quelqu'un ou quelque chose à l'extérieur du système.

N.B : un cas d'utilisation est un verbe à l'infinitif + groupe nominal

Formalisme :



- **La relation d'association :**

Elle exprime l'interaction existant entre un acteur et un cas d'utilisation.

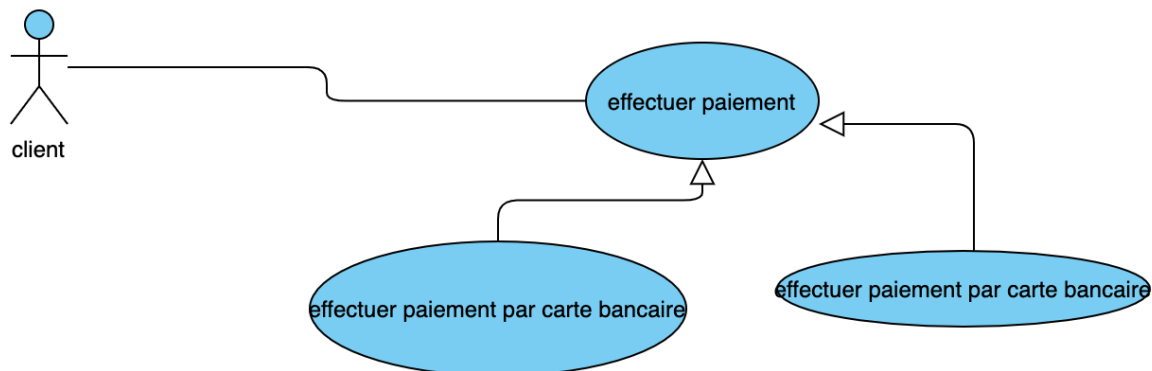
Formalisme :



Il existe 3 types de relations entre cas d'utilisation :

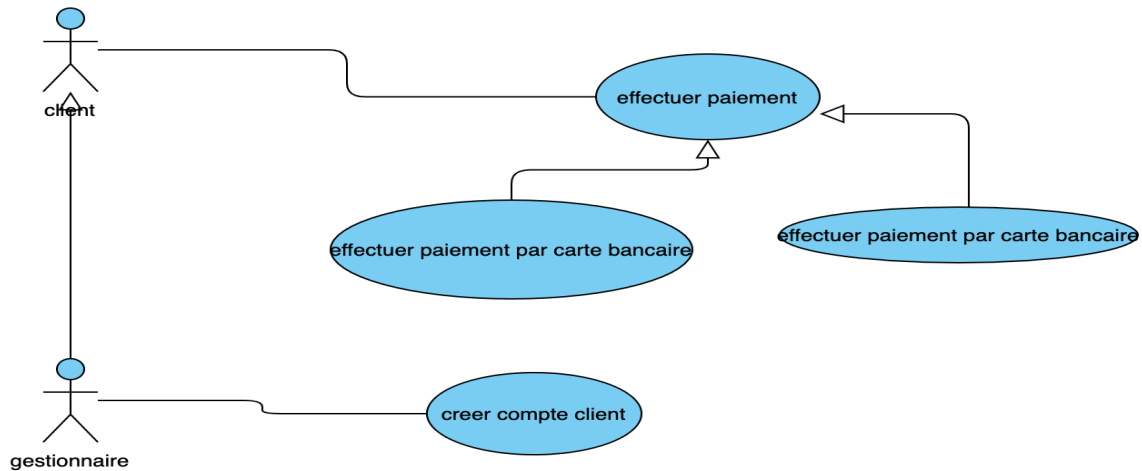
- la relation de généralisation : Dans une relation de généralisation entre 2 cas d'utilisation, le cas d'utilisation enfant est une spécialisation du cas d'utilisation parent. Cette relation est utile pour montrer qu'un cas d'utilisation est un type spécial d'un autre : le cas d'utilisation le plus spécialisé diffère quelque peu de l'original. Ainsi, toutes les étapes du cas d'utilisation original doivent être exécutées, certaines étant remplacées par des « versions » plus spécialisées.

Formalisme :

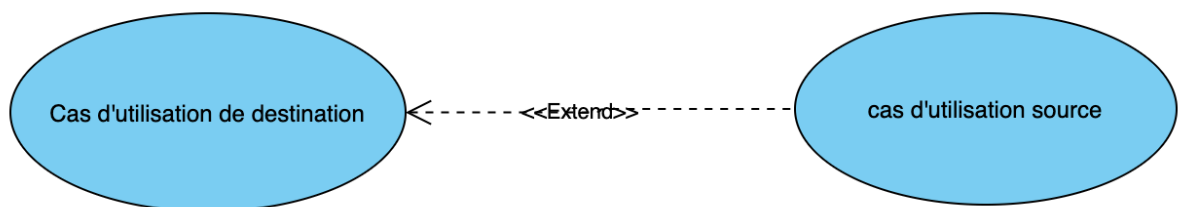


NB : un acteur peut également participer à des relations de généralisation avec d'autres acteurs. Les acteurs « enfant » seront alors capables de communiquer avec les mêmes cas d'utilisation que les acteurs « parents ».

Formalisme :

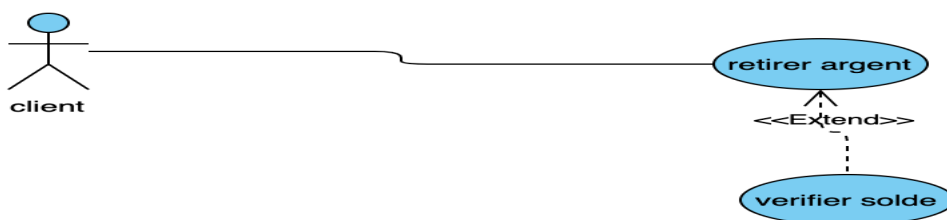


- la relation d'extension : Elle indique que le cas d'utilisation source ajoute son comportement au cas d'utilisation destination. Une relation d'extension est une ligne tiretée avec une pointe de flèche ouverte dirigée du cas d'utilisation source vers le cas d'utilisation de base. La flèche est libellée avec le mot clé «extend».



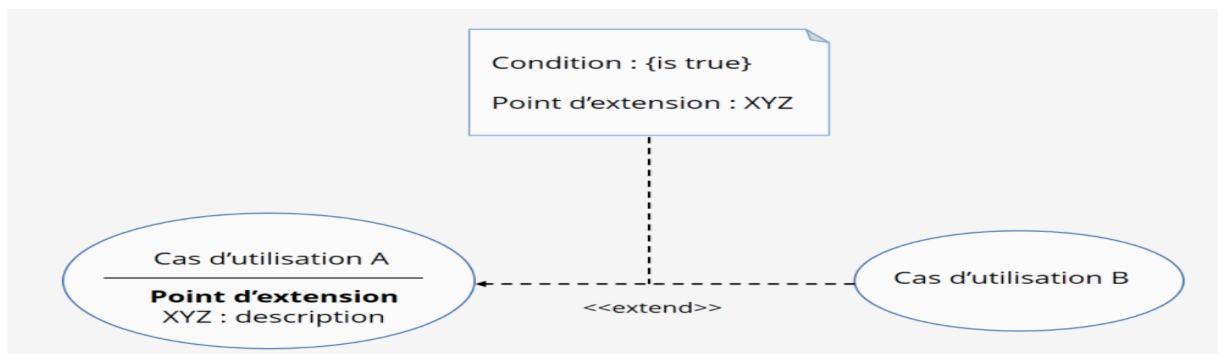
On peut utiliser une relation d'extension pour spécifier que le cas d'utilisation source étend le comportement d'un autre cas d'utilisation de destination c'est-à-dire lui ajoute des fonctionnalités supplémentaires.

NB : le cas d'utilisation source a un caractère facultatif.



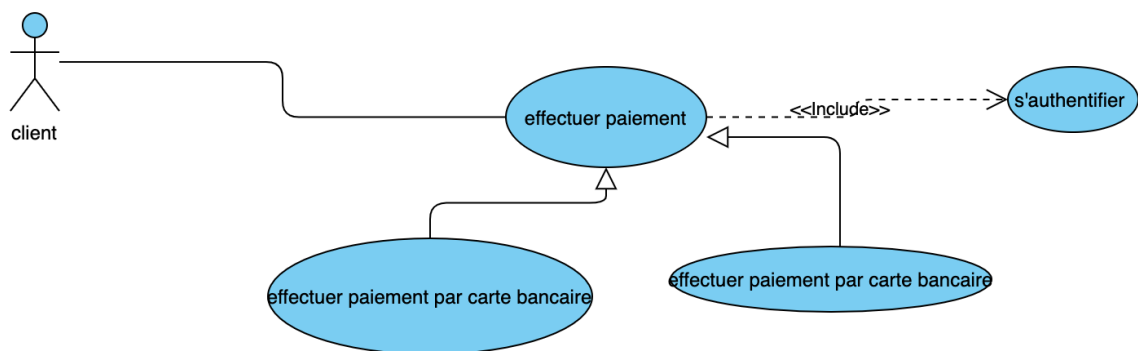


L'extension peut être soumise à condition. Le comportement ajouté est inséré au niveau d'un point d'extension défini dans le cas d'utilisation destination. Cette relation permet de modéliser les variantes de comportement d'un cas d'utilisation.

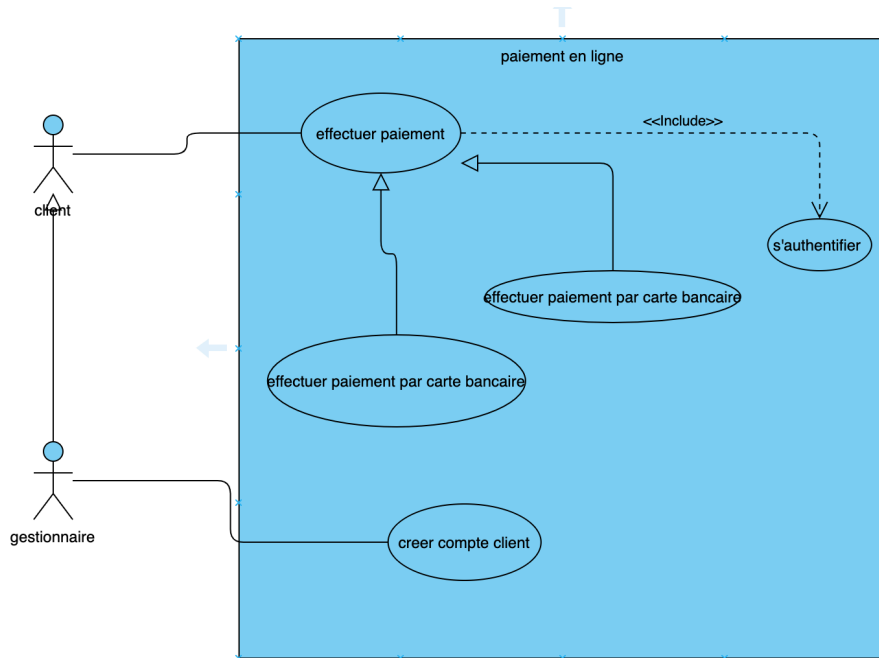


NB : un cas d'utilisation peut avoir plusieurs points d'extension.

- la relation d'inclusion : L'inclusion a un caractère obligatoire, la source spécifiant à quel endroit le cas d'utilisation cible doit être inclus.



- le système : il représente le périmètre auquel se rapporte les cas d'utilisation



Description textuelle des cas d'utilisation

La description d'un cas d'utilisation se fait par des scénarios qui définissent la suite logique des actions qui constituent ce cas d'utilisation. Cette description précise ce que fait l'acteur et ce que fait le système, elle clarifie ainsi le déroulement de la fonctionnalité en décrivant la chronologie des actions qui devront être réalisées. Celle-ci présente trois principales parties :

Partie 1 : Identification.

- Titre : Nom du cas d'utilisation
- Résumé : description du cas d'utilisation.
- Acteurs : descriptions des acteurs principaux et secondaires.
- Dates : Date de création et date de mise à jour.
- Responsable : Noms du ou des responsables.
- Version : Numéro de la version.

Partie 2 : Description des scénarios

- Les préconditions : État du système avant que le cas d'utilisation puisse être déclenché.
- Les Scénarios : Il y a plusieurs types de scénarios :



- ♣ Le scénario nominal qui correspond à un déroulement normal d'un cas d'utilisation.
- ♣ Les scénarios alternatifs qui sont des variantes du scénario nominal. C'est le cas des étapes liées à des conditions.
- ♣ Les scénarios d'exceptions : On parlera de scénario d'exception lorsqu'une étape du processus pourrait être perturbée à cause d'un événement anormal.
- Les post-conditions : Elles décrivent l'état du système après l'issue de chaque scénario.

Partie 3 : Exigence non fonctionnelle : fiabilité, performance..

Exemple : description textuelle du cas d'utilisation s'authentifier

- Titre : S'authentifier
- Résumé : ce cas d'utilisation permet aux utilisateurs d'avoir accès à la plateforme
- Acteurs : -acteur principal : tous les utilisateurs
- Dates : Date de création : 24 novembre 2020
- Responsable : Belinga Estelle
- Version : 1.0
- Préconditions : L'utilisateur a un compte sur la plateforme
- Scénario nominal:
 1. l'utilisateur saisit l'URL de la plateforme
 2. le système affiche le formulaire de connexion
 3. l'utilisateur saisit son login et son mot de passe
 4. le système vérifie la conformité des champs
 5. le système envoie les données à la BD
 6. la BD renvoie le résultat de la requête
 7. le système affiche un message de succès à l'utilisateur
- Scénario alternatif :
 - 4.a à l'étape 4 du scénario nominal, l'utilisateur entre des informations non correspondantes ou manquantes
 - 4.b le système affiche un message d'erreur puis retour à l'étape 2 du scénario nominal



6.a à l'étape 6 du scénario nominal si aucune correspondance n'est trouvée dans la bd, le système affiche un message d'erreur puis retour à l'étape 2 du scénario nominal.

- Scénario d'exception :

3.a' l'utilisateur saisit des identifiants erronés ou manquants à plus de trois reprises

3.b' le système part en veille pour 15 secondes puis retour à l'étape 2 du scénario nominal.

- Post condition de succès : l'utilisateur a accès à son espace de travail
- Post condition d'échec : l'utilisateur n'a pas accès à la plateforme
- Exigence non fonctionnelle : la saisie du mot de passe ne doit pas être visible à l'écran.

Cas pratique :

A la société camerounaise des banques, le processus de gestion de l'obtention des cartes bancaires visa se déroule ainsi qu'il suit : un demandeur désirant obtenir une carte visa doit en faire la sollicitation auprès d'un gestionnaire de compte. la carte n'est pas accordée si le demandeur n'est pas un client de la banque. Chaque jour, le bureau des cartes bancaires reçoit les demandes, les identifie, traite, classe et les transmet au service de gestion des cartes pour vérification de la conformité des informations envoyées par les clients. A la fin du mois, il a la charge de s'assurer de l'envoi des listes des demandeurs vers le centre international des cartes bancaires(CICB) pour établissement. Dès que la banque a reçu l'ensemble des cartes en provenance du centre, le bureau des cartes adresse au client via son gestionnaire, un avis de mise à disposition et un avis de prélèvement de la cotisation annuelle. Le client vient alors retirer sa carte et signe un accusé de réception. Si au bout de deux mois la carte n'a pas été retirée, elle est annulée.

6.1.2 Le diagramme d'activité

Le diagramme d'activité permet de mettre l'accent sur les traitements et de formaliser graphiquement la séquence d'actions réalisées dans un cas d'utilisation.



Éléments composants un diagramme d'activité

- **Une action:** c'est une opération élémentaire et instantanée. C'est une étape discrète à partir de laquelle se construit un comportement.

Formalisme :

saisir login et mot de passe

NB : il est à noter qu'il existe plusieurs types d'actions, il faut également noter que : si une action du cas d'utilisation correspond à l'appel d'un cas d'utilisation interne elle est représentée par une action contenant un signe spécial : deux cercles reliés par un trait.

Inscription comme client

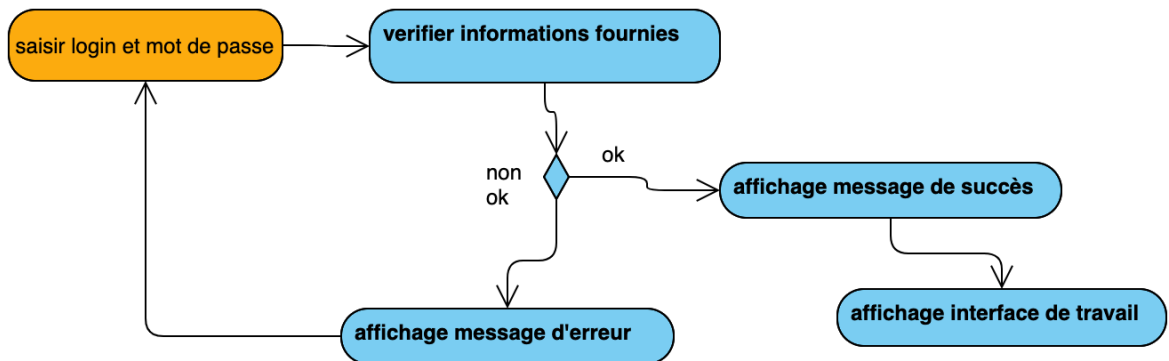
- Une activité : définit un comportement décrit par une série organisée d'unités dont les éléments simples sont les actions. Une activité est une opération complexe qui est décomposable en actions.
- **La transition:** elle représente le passage d'une action à une autre. Sa notation est une ligne avec une pointe de flèche.

Formalisme :

affichage formulaire

saisir login et mot de passe

- **Le nœud de décision:** C'est un nœud de contrôle qui permet de faire un choix entre plusieurs flots sortants. Il possède un arc entrant et plusieurs arcs sortants, accompagnés de conditions de garde pour conditionner le choix.



- **Le nœud initial :** Un nœud initial est un nœud de contrôle à partir duquel le flot débute. Il est représenté par un petit cercle plein.



- **Le nœud final :** Un nœud final est un nœud de contrôle(nœud utilisé pour coordonner une activité) possédant un ou plusieurs arcs entrants et aucun arc sortant. Il existe deux types de nœuds finaux: nœud de fin d'activité et nœud de fin de flot.

Lorsque l'un des arcs d'un nœud de fin d'activité est activé, l'exécution de l'activité enveloppante s'arrête et tous les nœuds ou flots actifs appartenant de cette activité sont abandonnés. Un nœud de fin d'activité est représenté par un cercle contenant un petit cercle plein.



Un flot est terminé lorsqu'un flot d'exécution atteint un nœud de fin de flot. Mais cette fin de flot n'a pas d'incidence sur les autres flots actifs de l'activité enveloppante. Un nœud de fin de flot est représenté par un cercle vide barré d'une croix.

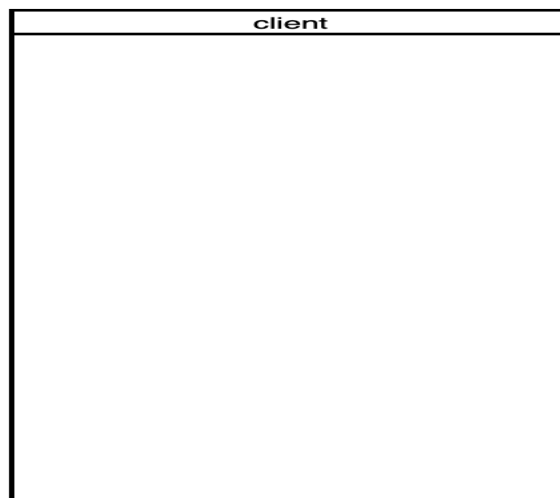




- Nœud de fusion : c' est un nœud de contrôle rassemblant plusieurs flots alternatifs entrants en un seul flot sortant. On ne l'utilise pas pour synchroniser des flots concurrents mais pour accepter un flot parmi plusieurs. Un nœud de fusion est représenté par un losange comme le nœud de décision.
 - Nœud de bifurcation : c'est un nœud de contrôle qui sépare un flot ou plusieurs flots concurrents. Il possède donc un arc entrant ou plusieurs arcs sortants. Il est représenté par un trait plein épais.
-



- Nœud d'union : C'est un nœud de contrôle qui synchronise des flots multiples. Il possède plusieurs arcs entrants et un seul arc sortant. Lorsque tous les arcs entrants sont activés, l'arc sortant l'est aussi également. Un nœud d'union est aussi représenté par un trait plein épais.
- Les partitions ou couloirs : ont pour but d'organiser les noeuds d'activités disposés dans un diagramme d'activités par le biais de regroupements. Graphiquement, les partitions sont représentées par des lignes continues. Elles peuvent prendre la forme d'un tableau. De plus, les noeuds d'activités doivent appartenir à une seule et unique partition et les transitions peuvent passer à travers les frontières des partitions.



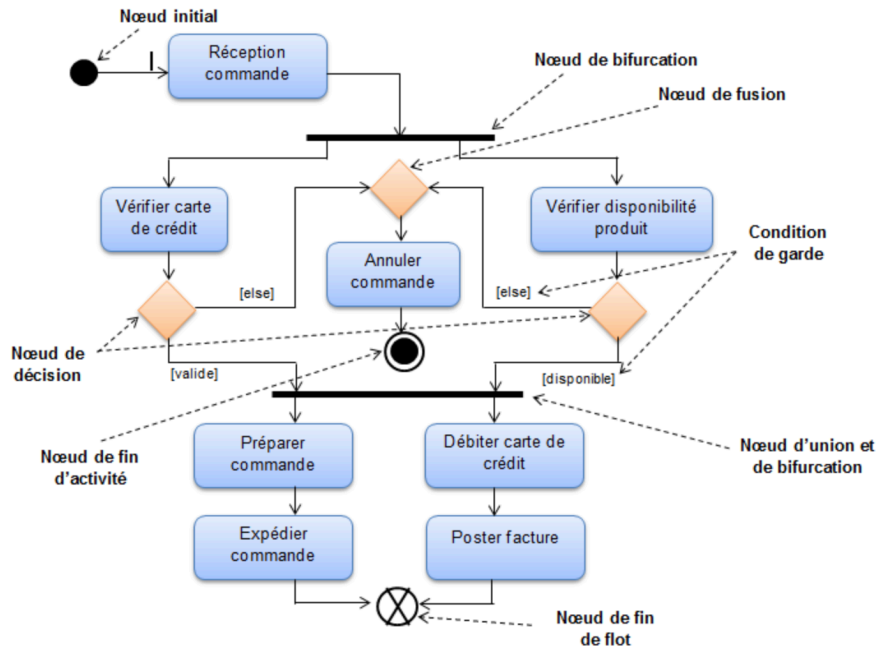
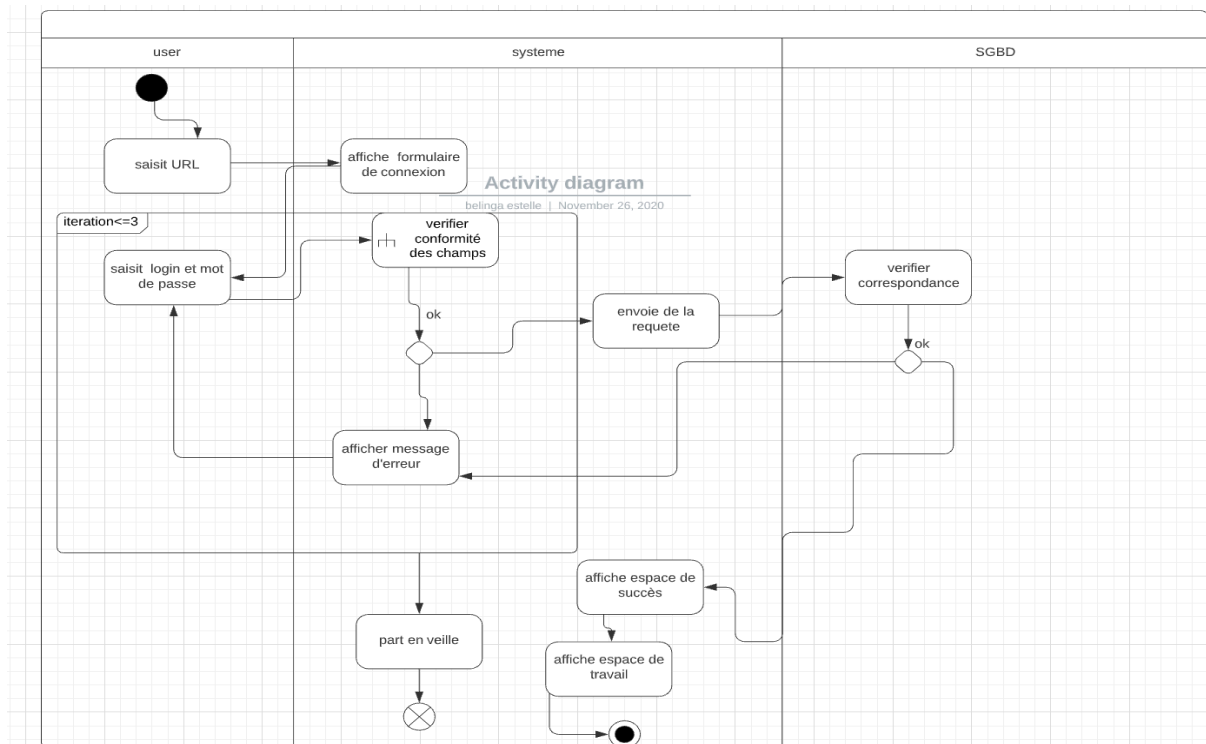


Diagramme d'activité du cas d'utilisation s'authentifier





6.1.3 Diagramme de communication ou collaboration

Semblable à un diagramme de séquence, les diagrammes de communication modélisent un transfert de message à l'aide de lignes de vie en numérotant les séquences. Ces numéros représentent l'ordre dans lequel les messages sont envoyés.

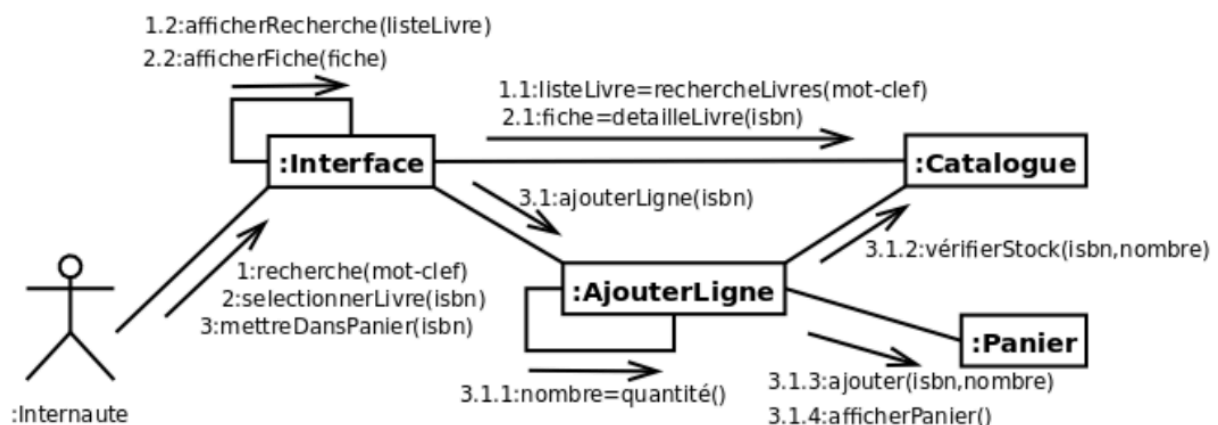
Sur les diagrammes de communication, les objets sont présentés avec des connecteurs d'association entre eux. Les messages sont ajoutés à ces associations et les flèches montrent la direction du flux de messages.

Éléments de modélisation d'un diagramme de communication

- La ligne de vie : est représentée par un rectangle contenant le nom et le type de l'instance.



- Les connecteurs : Les relations entre les lignes de vie sont appelées connecteurs et se représentent par un trait plein reliant deux lignes de vie.
- Les messages : désignent une communication particulière entre les lignes de vie, et sont généralement ordonnés selon un ordre de numéro croissant.



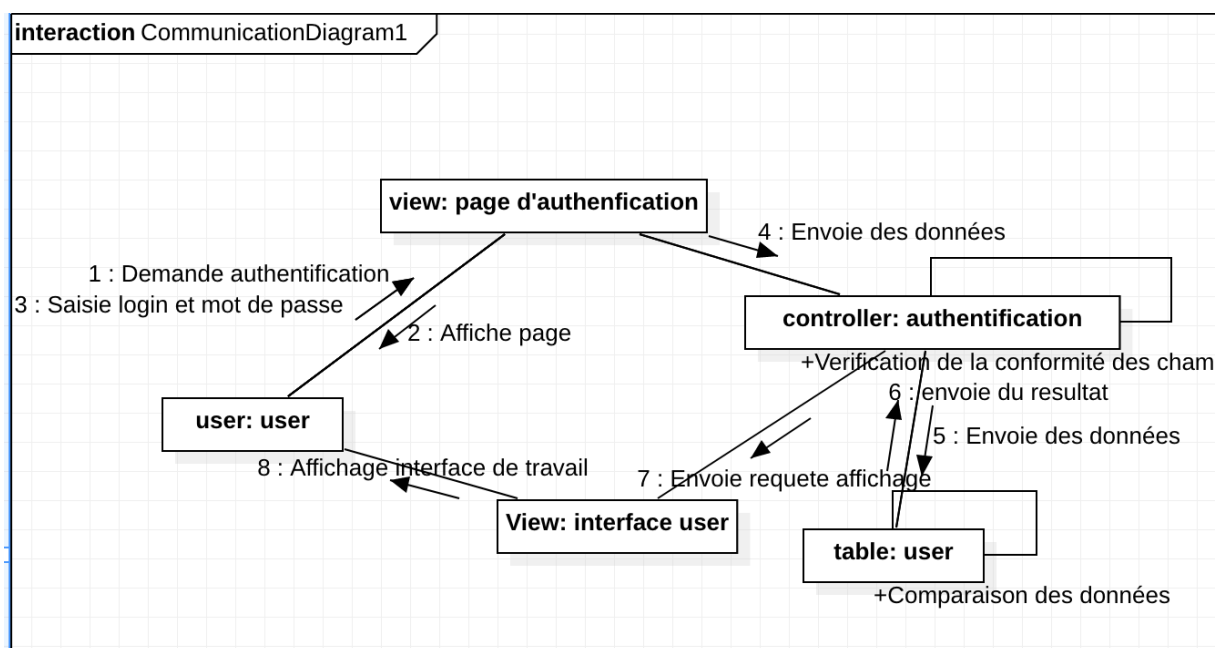


La principale difficulté de cette approche consiste à identifier les objets appropriés afin de transcrire les actions du cas d'utilisation. On préconise généralement de séparer les objets en 3 types :

- Les objets Boundary sont utilisés par les acteurs lorsqu'ils communiquent avec le système, il peut s'agir de fenêtres, d'écrans, de boîtes de dialogue ou de menus.
- Les objets Entity représentent des données stockées telles qu'une base de données, des tables de base de données ou tout type d'objet temporaire tel qu'un résultat de recherche.
- Les objets Control sont utilisés pour contrôler les objets Boundary et Entity, et qui représentent un transfert d'informations.



Diagramme de communication du cas s'authentifier en fonction du scénario nominal





6.1.4 Diagramme de séquence

Il permet de représenter les échanges entre les différents objets et acteurs du système en fonction du temps et de manière chronologique. Il a pour but de :

- Représenter les détails d'un cas d'utilisation
- Modéliser le déroulement logique d'une procédure, fonction ou opération complexe
- Voir comment les objets et les composants interagissent entre eux pour effectuer un processus.
- Schématiser et comprendre le fonctionnement détaillé d'un scénario.

Éléments modélisant un diagramme de séquence

- Ligne de vie : La ligne de vie représente le **déroulement temporel d'un processus**. C'est une ligne verticale en pointillés qui montre les événements séquentiels affectant un objet au cours du processus schématisé.



- L'objet : C'est l'instance d'une classe, et est rangé horizontalement.



- L'Acteur : Il peut communiquer avec des objets, et est énuméré en colonne. Un acteur est modélisé en utilisant le symbole habituel :



- L'Activation : Représente le temps nécessaire pour qu'un objet ou un acteur accomplisse une tâche, elle indique quand l'objet effectue une action. Plus la tâche nécessite de temps, plus la boîte d'activation est longue.





- Message : modélisé par une flèche horizontale entre les activations, il indique les communications entre les objets. Il existe plusieurs types de messages dans un diagramme de séquence :
 - Les messages synchrones : La réception d'un message synchrone doit provoquer chez le destinataire le lancement d'une de ses méthodes ou opérations. L'expéditeur du message reste bloqué pendant toute l'exécution de la méthode et attend donc la fin de celle-ci avant de pouvoir lancer un nouveau message. C'est le message le plus fréquemment utilisé. Il est représenté par une ligne avec une pointe de flèche pleine.



- Les messages asynchrones : ils ne nécessitent pas de réponse du destinataire, avant que l'expéditeur ne continue. Ils sont représentés par une ligne terminée par une pointe de flèche.



- Les messages de retour : ils représentent la réponse à un message synchrone. Ils sont représentés par des flèches tiretées.



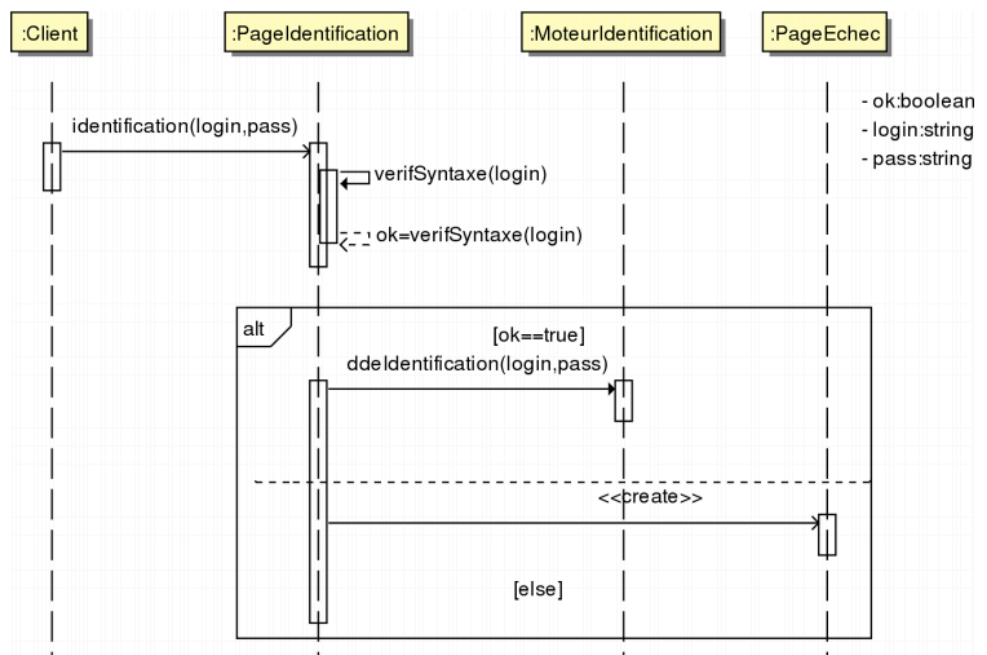
- Les fragments d'interaction combinés : Ils représentent une articulation d'interactions. Il est défini par un opérateur et des opérandes. L'opérateur conditionne la signification du fragment. Il existe 12 opérateurs définis dans la notation UML 2.0. Les fragments combinés permettent de décrire des diagrammes de séquence de manière compacte. Un fragment combiné est représenté par un rectangle dont le coin supérieur gauche contient



un pentagone. Dans le pentagone figure le type de la combinaison, appelé *opérateur d'interaction*.

La liste suivante regroupe les opérateurs d'interaction par fonctions :

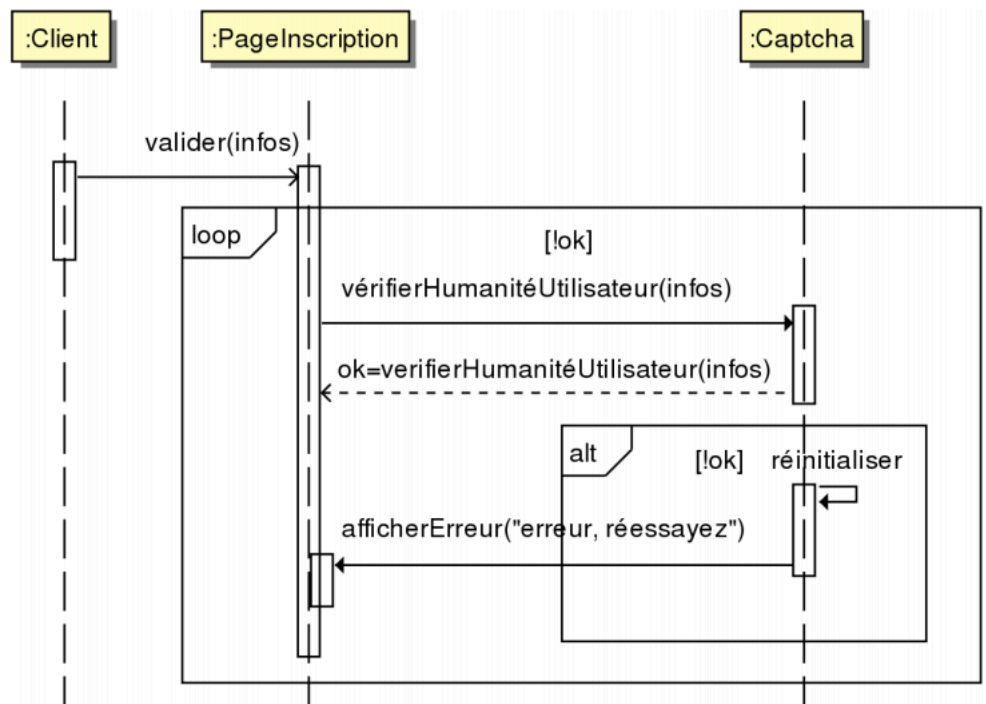
- les opérateurs de choix et de boucle : **alternative**, **option**, **break** et **loop**.
L'opérateur « alt » désigne un choix, une alternative. Il représente deux comportements possibles : c'est en quelque sorte l'équivalent du **SI...ALORS...SINON** : donc, une seule des deux branches sera réalisée dans un scénario donné. La condition d'exécution d'une des deux branches (l'équivalent du SI) peut être explicite ou implicite. L'utilisation du sinon permet d'indiquer que la branche est exécutée si la condition du alt est fausse.



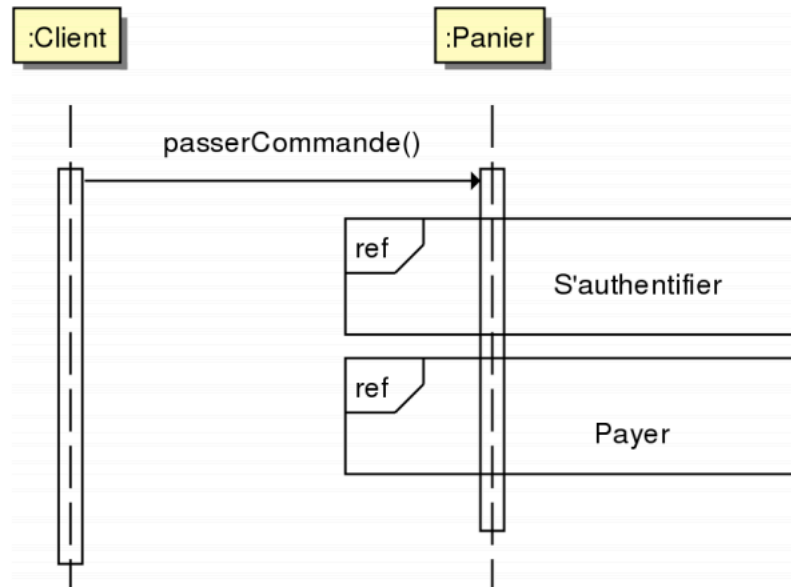
L'opérateur « opt » désigne un fragment combiné optionnel comme son nom l'indique : c'est-à-dire qu'il représente un comportement qui peut se produire... ou pas. Un fragment optionnel est équivalent à un fragment « alt » qui ne posséderait pas d'opérande else (qui n'aurait qu'une seule branche). Un fragment optionnel est donc une sorte de **SI...ALORS**.



L'opérateur « Loop » (boucle) est noté « loop ». Cet opérateur est utilisé pour décrire un ensemble d'interactions qui s'exécutent **en boucle**. En général, une contrainte appelée **garde** indique le nombre de répétitions (minimum et maximum) ou bien une condition booléenne à respecter.



- les opérateurs contrôlant l'envoi en parallèle de messages : **parallel** et **critical region** ;
- les opérateurs contrôlant l'envoi de messages : **ignore**, **consider**, **assertion** et **negative** ;
- les opérateurs fixant l'ordre d'envoi des messages : **weak sequencing** , **strict sequencing**.
- L'opérateur de réutilisation de séquences : ref qui permet d'indiquer la réutilisation d'un diagramme de séquences.



Exemple du diagramme de séquence de s'authentifier

6.2 Les diagrammes structurels ou statiques

Ils Décrivent la structure du système en termes de Composants du système : Objets, Classes, Paquetages, Composants, et de relations entre ces composants ; Spécialisation, Association, Dépendance, ...

6.2.1 le diagramme de paquetage

Il permet de décomposer le système en catégories ou parties plus facilement observables appelés « packages». L'utilisation la plus courante pour les diagrammes de paquetages est d'organiser des Diagrammes de Cas d'Utilisation et des Diagrammes de Classes. Bien que l'Utilisation des Diagrammes de Paquetages ne se limite pas à ces éléments UML.

Il est représenté par un dossier avec son nom à l'intérieur.



6.2.2 Le diagramme de classe

Le diagramme de classes exprime la structure statique du système en termes de classes et de relations entre ces classes. L'intérêt du diagramme de classe est de modéliser les entités du



système d'information. Le diagramme de classe permet de représenter l'ensemble des informations finalisées qui sont gérées par le domaine en décrivant ce que les attributs et les comportements, qu'il a plutôt que de détailler les méthodes pour atteindre les opérations.

. Ces informations sont structurées, c'est-à-dire qu'elles ont regroupées dans des classes. Le diagramme met en évidence d'éventuelles relations entre ces classes. Le diagramme de classes comporte 6 concepts : classe, attribut, identifiant, relation, opération, généralisation / spécialisation.

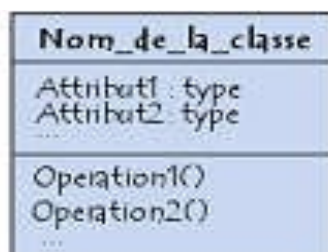
Éléments modélisant un diagramme de classe

- **La classe** : est une description abstraite d'un ensemble d'objets du domaine de l'application : elle définit leur structure, leur comportement et leurs relations.

Représentation : les classes sont représentées par des rectangles compartimentés :

- le 1er compartiment représente le nom de la classe
- le 2ème compartiment représente les attributs de la classe
- le 3ème compartiment représente les opérations de la classe

Formalisme :



- **L'attribut** : Une classe correspond à un concept global d'information et se compose d'un ensemble d'informations élémentaires, appelées attributs de classe. Un attribut représente la modélisation d'une information élémentaire représentée par son nom et son format.

UML définit 3 niveaux de visibilité pour les attributs :

- public (+) : l'élément est visible pour tous les clients de la classe
- protégé (#) : l'élément est visible pour les sous-classes de la classe
- privé (-) : l'élément n'est visible que par les objets de la classe dans laquelle il est déclaré.



Formalisme :

NOM CLASSE		
+	Attribut public	: int
#	Attribut protégé	: int
-	Attribut privé	: int

- **l'identifiant** : L'identifiant est un attribut particulier, qui permet de repérer de façon unique chaque objet, instance de la classe.
- **L'opération** : représente un élément de comportement des objets, défini de manière globale dans la classe. Une opération est une fonctionnalité assurée par une classe. La description des opérations peut préciser les paramètres d'entrée et de sortie ainsi que les actions élémentaires à exécuter.

Formalisme :

FACTURE		
+	No facture	: int
+	Date	: Date
+	Montant	: double
+	/ Montant TVA	: double
+	Op publique ()	
#	Op protégée ()	
-	Op privée ()	

- **La relation** : Il existe plusieurs types de relations entre classes : l'association, la généralisation/spécialisation, la dépendance.
- **l'association** : L'association est la relation la plus courante et la plus riche du point de vue sémantique. Une association est une relation entre deux classes (association binaire) ou plus (association n-aire), qui décrit les connexions structurelles entre leurs instances. Une association indique donc qu'il peut y avoir des liens entre des instances des classes associées.
 - Une association binaire est matérialisée par un trait plein entre les classes associées. Elle peut être ornée d'un nom, avec éventuellement une précision du



sens de lecture (► ou ◄). Quand les deux extrémités de l'association pointent vers la même classe, l'association est dite *réflexive*.

- Une association n-aire lie plus de deux classes. Elle est représentée par une association n-aire par un grand losange avec un chemin partant vers chaque classe participante. Le nom de l'association, le cas échéant, apparaît à proximité du losange.

Dans une association, on retrouve la notion de multiplicité qui définit le nombre d'instances de l'association pour une instance de la classe. La multiplicité est définie par un nombre entier ou un intervalle de valeurs. La multiplicité est notée sur le rôle (elle est notée à l'envers de la notation MERISE).

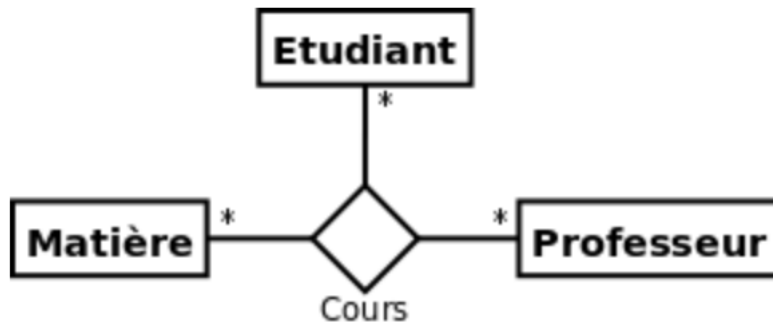
1	Un et un seul
0..1	Zéro ou un
N ou *	N (entier naturel)
M..N	De M à N (entiers naturels)
0..*	De zéros à plusieurs
1..*	De 1 à plusieurs

Formalisme :



Interprétation :

- Une instance de la classe société emploie au moins 0 Personne et au plus plusieurs instances de la classe personne.
- Une instance de la classe personne peut être employé par au moins 0 société et au plus plusieurs sociétés.



Interprétation :

Une instance d'étudiant est enseigné par au moins un et au plus plusieurs professeurs.

Une instance d'étudiant suit au moins 0 matière et au plus plusieurs matières

Une instance de matière est enseigné par au moins un et au plus plusieurs professeurs.

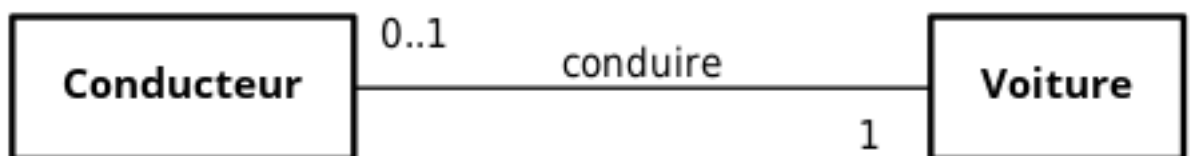
Une instance de matière est suivie par au moins un étudiant et au plus plusieurs étudiants

Une instance de professeur peut enseigné au moins une matière et au plus plusieurs matières

Une instance de professeur peut enseigner au moins un étudiant et au plus plusieurs étudiants.

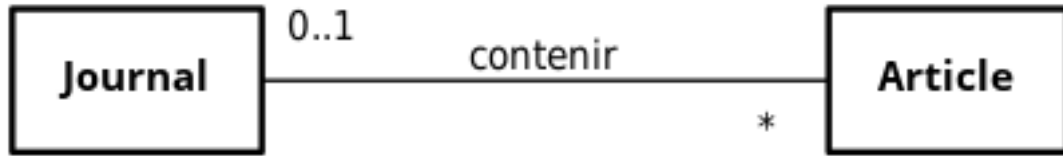
Les associations sont classées en trois catégories en fonction des multiplicités apposées sur celles-ci :

1. **un à un (one-to-one) :**



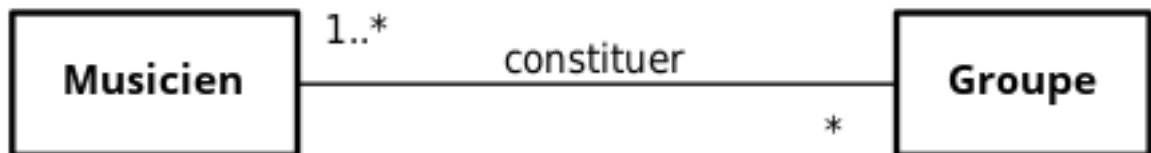
- Un *conducteur* conduit une et **une seule** *voiture* à la fois
- Une *voiture* n'est pas conduite (en stationnement) ou conduite par **un seul** *conducteur* à la fois

2. **un à plusieurs (one-to-many) ou plusieurs à un (many-to-one) :**



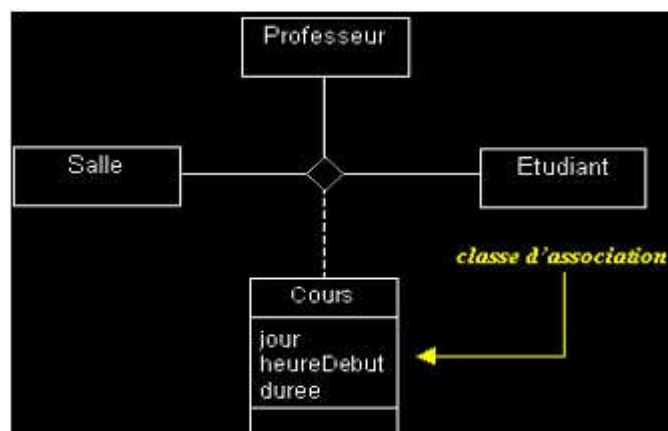
- Un *journal* contient aucun (journal en préparation), un ou **plusieurs** *articles*
- Un *article* est contenu dans aucun (en cours d'écriture) ou **un seul** *journal*

3. **plusieurs à plusieurs (many-to-many) :**



- Un *musicien* fait partie d'aucun, un ou **plusieurs** *groupes*
- Un *groupe* est constitué de un ou **plusieurs** *musiciens*

- La classe d'association : Les attributs d'une classe dépendent fonctionnellement de l'identifiant de la classe. Parfois, un attribut dépend fonctionnellement de 2 identifiants, appartenant à 2 classes différentes. Par exemple, l'attribut « note » dépend fonctionnellement du code de la matière et du matricule de l'étudiant. On va donc placer l'attribut « note » dans l'association « composer ». Dans ce cas, l'association est dite « porteuse d'attributs ».





- L'agrégation : Dans les diagrammes de classes, nous sommes souvent appelés à utiliser des associations que nous nommerions contient ou appartient à. En fait, cet usage est tellement fréquent que UML a défini une notation spécifique pour ce type d'association : l'agrégation. Une agrégation exprime une relation « contenant / contenu » ou « ensemble / élément », ou encore « agrégé / agrégat ». Elle se représente toujours par un petit losange du côté de l'agrégat.

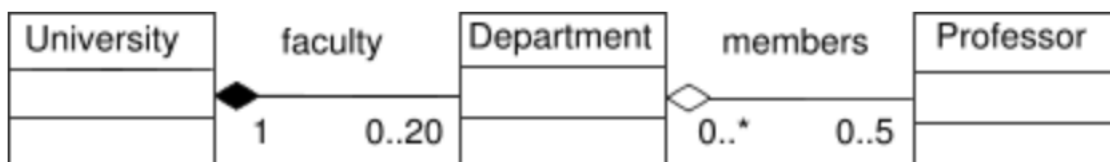
Règles permettant de choisir une agrégation :

- Est-ce une partie de ?
- Les opérations appliquées à l'ensemble sont-elles appliquées à l'élément ?
- Les changements d'états sont-ils liés ?



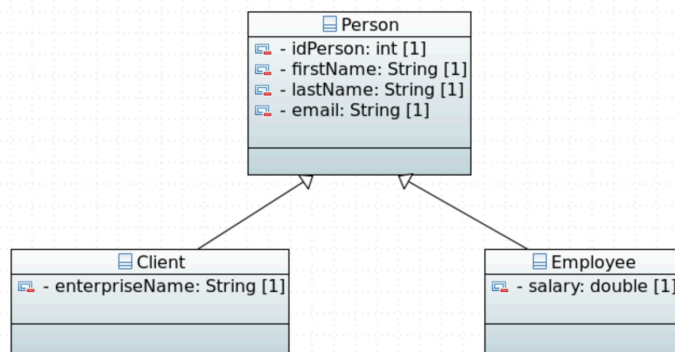
N.B :

- Un élément agrégé peut être lié à d'autres classes
- La suppression de l'ensemble n'entraîne pas celle de l'élément
- Un agrégat peut être multiple.





- La composition : La composition est un cas particulier de l'agrégation dans laquelle la vie des composants est liée à celle des agrégats. Elle fait souvent référence à une contenance physique. Dans la composition l'agrégat ne peut être multiple. La composition implique, en plus de l'agrégation, une coïncidence des durées de vie des composants : la destruction de l'agrégat (ou conteneur) implique automatiquement la destruction de tous les composants liés. La composition se représente par un petit losange noir du coin de l'agrégat.
- La généralisation / spécialisation : Le principe de généralisation / spécialisation permet d'identifier parmi les objets d'une classe (générique) des sous-ensembles d'objets (des classes spécialisées) ayant des définitions spécifiques. La classe plus spécifique (appelée aussi classe fille, classe dérivée, classe spécialisée, classe descendante ...) est cohérente avec la classe plus générale (appelée aussi classe mère, classe générale ...), c'est-à-dire qu'elle contient par héritage tous les attributs, les membres, les relations de la classe générale, et peut contenir d'autres. Une relation de généralisation est indiquée par une flèche creuse se dirigeant vers la classe "parent".



Effectuer le diagramme de classe du cas pratique



6.2.3 le diagramme de composant

Le diagramme des composants est principalement employé pour décrire les dépendances entre les divers composants logiciels tels que la dépendance entre les fichiers exécutables et les fichiers sources.

Éléments modélisant un diagramme de composant

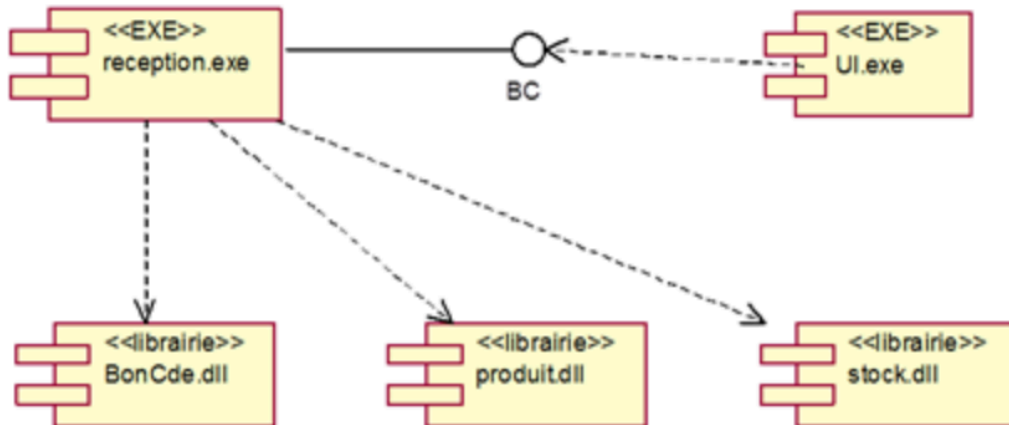
- Composants : Un composant représente une entité logicielle d'un système. Un composant est représenté par une boîte rectangulaire, avec deux rectangles dépassant du côté gauche.



UML définit 5 stéréotypes aux composants :

- « document » : un document quelconque,
 - « exécutable » : un programme qui peut s'exécuter,
 - « fichier » : un document contenant un code source ou des données,
 - « bibliothèque » : une bibliothèque statique ou dynamique,
 - « table » : une table de base de données relationnelle.
- Dépendance : Une dépendance est utilisée pour modéliser la relation entre deux composants. La notation utilisée pour cette relation de dépendance est une flèche pointillée, se dirigeant d'un composant donné au composant dont il dépend.





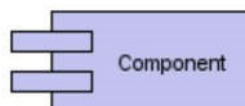
6.2.4 Le diagramme de déploiement

Le diagramme de déploiement modélise les composants matériels utilisés pour implémenter un système et l'association entre ces composants. Des diagrammes de déploiement peuvent être mise en œuvre dès la phase de conception pour documenter l'architecture physique du système.

Éléments modélisant un diagramme de déploiement

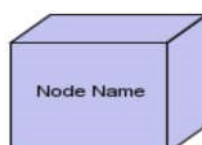
- Composant

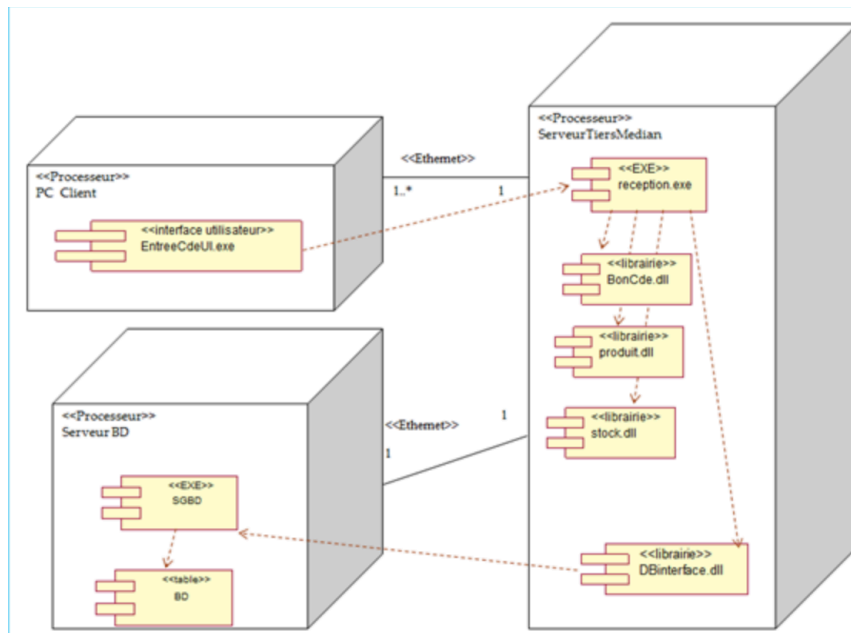
Un composant représente une entité logicielle du système. (Fichier de code source, programmes, documents, fichiers de ressource, etc.). Sur un diagramme de déploiement, les composants sont placés dans des nœuds pour identifier l'endroit de leur déploiement.



- Nœud

Un nœud représente un ensemble d'éléments matériels du système. Cette entité est représentée par un cube tridimensionnel.





7. Le processus UP

La maîtrise des processus de développement implique une organisation et un suivi des activités : c'est ce à quoi s'attachent les différentes méthodes qui s'appuient sur l'utilisation du langage UML pour modéliser un système d'information. UP (Unified Process) est une méthode générique de développement de logiciel. Générique signifie qu'il est nécessaire d'adapter UP au contexte du projet, de l'équipe, du domaine et/ou de l'organisation (exemple : R.UP, X.UP, 2 TUP...).

7.1 Les caractéristiques d'UP

Le processus unifié est un processus de développement logiciel : il regroupe les activités à mener pour transformer les besoins d'un utilisateur en système logiciel.

Les caractéristiques essentielles du processus unifié sont :

- Il est à base de composants,
- Il utilise le langage UML (ensemble d'outils et de diagramme),
- Il est piloté par les cas d'utilisation,

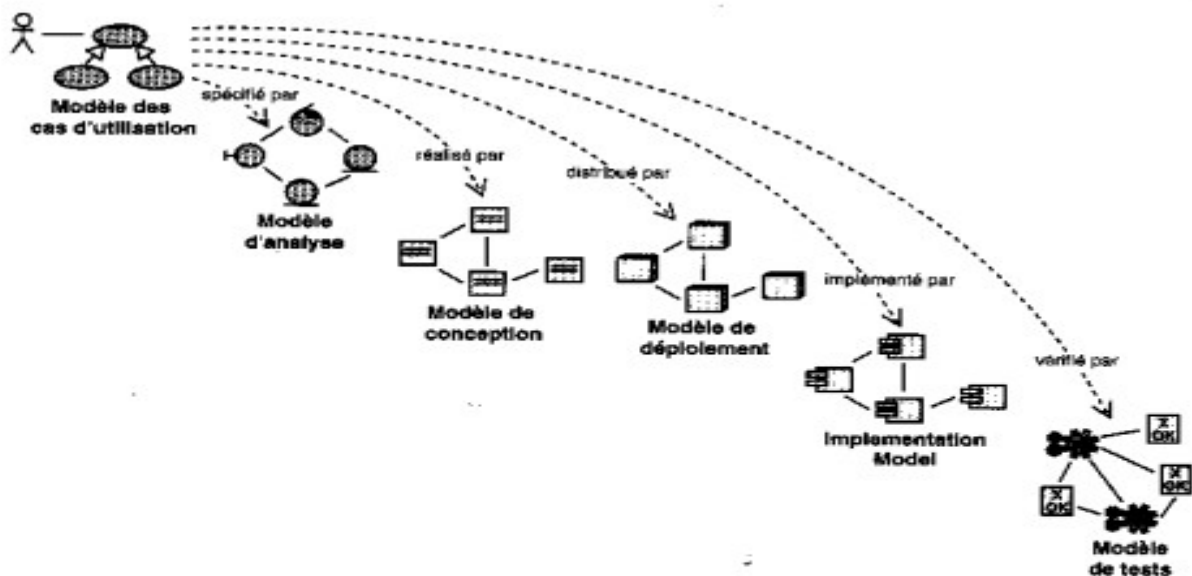


- Il est centré sur l'architecture, - Il est itératif et incrémental.

- Le processus unifié est piloté par les cas d'utilisation

L'objectif principal d'un système logiciel est de rendre service à ses utilisateurs ; il faut par conséquent bien comprendre les désirs et les besoins des futurs utilisateurs. Le processus de développement sera donc centré sur l'utilisateur. Le terme utilisateur ne désigne pas seulement les utilisateurs humains mais également les autres systèmes.

Les cas d'utilisation ne sont pas un simple outil de spécification des besoins du système. Ils vont complètement guider le processus de développement à travers l'utilisation de modèles basés sur l'utilisation du langage UML.



- A partir du modèle des cas d'utilisation, les développeurs créent une série de modèles de conception et d'implémentation réalisant les cas d'utilisation.
 - Chacun des modèles successifs est ensuite révisé pour en contrôler la conformité par rapport au modèle des cas d'utilisation.
 - Enfin, les testeurs testent l'implémentation pour s'assurer que les composants du modèle d'implémentation mettent correctement en œuvre les cas d'utilisation.
- Le processus unifié est centré sur l'architecture



Dès le démarrage du processus, on aura une vue sur l'architecture à mettre en place. L'architecture d'un système logiciel peut être décrite comme les différentes vues du système qui doit être construit. L'architecture logicielle équivaut aux aspects statiques et dynamiques les plus significatifs du système. L'architecture émerge des besoins de l'entreprise, tels qu'ils sont exprimés par les utilisateurs et autres intervenants et tels qu'ils sont reflétés par les cas d'utilisation.

Elle subit également l'influence d'autres facteurs :

- la plate-forme sur laquelle devra s'exécuter le système ;
 - les briques de bases réutilisables disponibles pour le développement ;
 - les considérations de déploiement, les systèmes existants et les besoins non fonctionnels (performance, fiabilité..).
-
- Le processus unifié est itératif et incrémental

Le développement d'un produit logiciel destiné à la commercialisation est une vaste entreprise qui peut s'étendre sur plusieurs mois. On ne va pas tout développer d'un coup. On peut découper le travail en plusieurs parties qui sont autant de mini projets. Chacun d'entre eux représentant une itération qui donne lieu à un incrément.

Une itération désigne la succession des étapes de l'enchaînement d'activités, tandis qu'un incrément correspond à une avancée dans les différents stades de développement. Le choix de ce qui doit être implémenté au cours d'une itération repose sur deux facteurs :

- Une itération prend en compte un certain nombre de cas d'utilisation qui ensemble, améliorent l'utilisabilité du produit à un certain stade de développement.
- L'itération traite en priorité les risques majeurs.

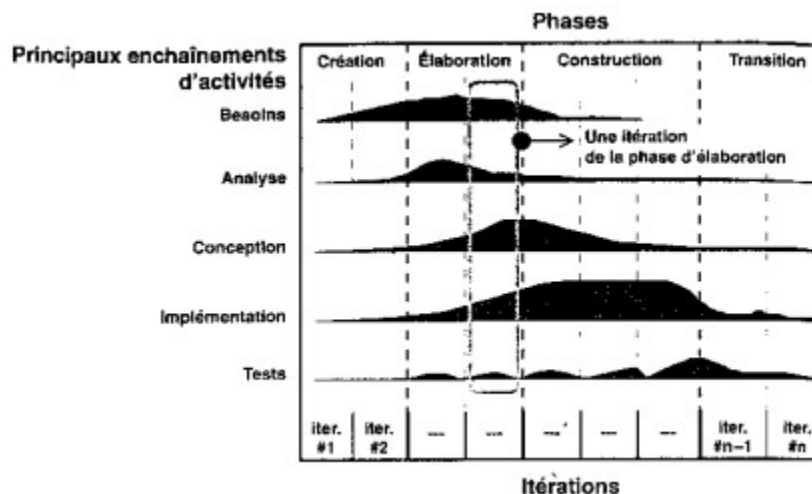
Un incrément constitue souvent un additif. A chaque itération, les développeurs identifient et spécifient les cas d'utilisations pertinents, créent une conception en se laissant guider par l'architecture choisie, implémentent cette conception sous forme de composants et vérifie que



ceux-ci sont conformes aux cas d'utilisation. Dès qu'une itération répond aux objectifs fixés le développement passe à l'itération suivante. Pour rentabiliser le développement il faut sélectionner les itérations nécessaires pour atteindre les objectifs du projet. Ces itérations devront se succéder dans un ordre logique. Un projet réussi suivra un déroulement direct, établi dès le début par les développeurs et dont ils ne s'éloigneront que de façon très marginale. L'élimination des problèmes imprévus fait partie des objectifs de réduction des risques.

7.2 Le cycle de vie d'UP

Présentation





Phase	Description et Enchaînement d'activités
Phase de création	Traduit une idée en vision de produit fini et présente une étude de rentabilité pour ce produit - Que va faire le système pour les utilisateurs ? - A quoi peut ressembler l'architecture d'un tel système ? - Quels sont l'organisation et les coûts du développement de ce produit ? On fait apparaître les principaux cas d'utilisation. L'architecture est provisoire, identification des risques majeurs et planification de la phase d'élaboration.
Phase d'élaboration	Permet de préciser la plupart des cas d'utilisation et de concevoir l'architecture du système. L'architecture doit être exprimée sous forme de vue de chacun des modèles. Émergence d'une architecture de référence . A l'issue de cette phase, le chef de projet doit être en mesure de prévoir les activités et d'estimer les ressources nécessaires à l'achèvement du projet.
Phase de construction	Moment où l'on construit le produit. L'architecture de référence se métamorphose en produit complet, elle est maintenant stable. Le produit contient tous les cas d'utilisation que les chefs de projet, en accord avec les utilisateurs ont décidé de mettre au point pour cette version. Celle-ci doit encore avoir des anomalies qui peuvent être en partie résolues lors de la phase de transition.
Phase de transition	Le produit est en version bêta. Un groupe d'utilisateurs essaye le produit et détecte les anomalies et défauts. Cette phase suppose des activités comme la fabrication, la formation des utilisateurs clients, la mise en œuvre d'un service d'assistance et la correction des anomalies constatées. (où le report de leur correction à la version suivante)

Description

Le processus unifié répète un certain nombre de fois une série de cycles. Tout cycle se conclut par la livraison d'une version du produit aux clients et s'articule en 4 phases : création, élaboration, construction et transition, chacune d'entre elles se subdivisant à son tour en itérations. Pour mener efficacement le cycle, les développeurs ont besoin de construire toutes les représentations du produit logiciel :



Modèle des cas d'utilisation	Expose les cas d'utilisation et leurs relations avec les utilisateurs
Modèle d'analyse	Détaille les cas d'utilisation et procède à une première répartition du comportement du système entre divers objets
Modèle de conception	Définit la structure statique du système sous forme de sous système, classes et interfaces ; Définit les cas d'utilisation réalisés sous forme de collaborations entre les sous systèmes les classes et les interfaces
Modèle d'implémentation	Intègre les composants (code source) et la correspondance entre les classes et les composants
Modèle de déploiement	Définit les nœuds physiques des ordinateurs et l'affectation de ces composants sur ces nœuds.
Modèle de test	Décrit les cas de test vérifiant les cas d'utilisation
Représentation de l'architecture	Description de l'architecture

7.3 Exemple de processus de développement logiciel qui implémente le Processus Unifié : 2TUP (Two tracks unified process)

Dans le processus 2TUP, les activités de développement sont organisées suivant 5 workflows qui décrivent :

- La capture des besoins,
- L'analyse,
- La conception,
- L'implémentation
- Et le test.

Le processus 2TUP est une trame des meilleures pratiques de développement, il doit être utilisé comme un guide pour réaliser un projet et non comme l'arme ultime et universelle de développement.

7.3.1 Le cycle de développement avec 2TUP

Le 2TUP propose un cycle de développement en Y, qui dissocie les aspects techniques des aspects fonctionnels. Il commence par une étude préliminaire qui consiste essentiellement à identifier les acteurs qui vont interagir avec le système à construire, les messages qu'échangent les acteurs et le système. En suite à produire le cahier des charges et à modéliser le contexte. Le processus s'articule autour de 3 phases



essentielles : une branche technique ; une branche fonctionnelle et une phase de réalisation.

- La branche fonctionnelle

La branche fonctionnelle capitalise la connaissance du métier de l'entreprise. Cette branche comporte :

- La capture des besoins fonctionnels, qui produit un modèle des besoins focalisé sur le métier des utilisateurs. Elle qualifie au plus tôt le risque de produire un système inadapté aux utilisateurs. De son côté, la maîtrise d'œuvre consolide les spécifications et en vérifie la cohérence et l'exhaustivité.
- L'analyse, qui consiste à étudier précisément la spécification fonctionnelle de manière à obtenir une idée de ce que va réaliser le système en termes de métier. Les résultats de l'analyse ne dépendent d'aucune technologie particulière.

- La branche technique

La branche technique capitalise un savoir-faire technique et/ou des contraintes techniques. Les techniques développées pour le système le sont indépendamment des fonctions à réaliser. Cette branche droite comporte :

- La capture des besoins techniques, qui recense toutes les contraintes et les choix dimensionnant la conception du système. Les outils et les matériels sélectionnés ainsi que la prise en compte de contraintes d'intégration avec l'existant conditionnent généralement des pré-requis d'architecture technique ;
- La conception générique, qui définit ensuite les composants nécessaires à la construction de l'architecture technique. Cette conception est complètement indépendante des aspects fonctionnels. Elle a pour objectif d'uniformiser et de réutiliser les mêmes mécanismes pour tout un système. L'architecture technique construit le squelette du système informatique et écarte la plupart des risques de niveau technique. L'importance de sa réussite est telle qu'il est conseillé de réaliser un prototype pour assurer sa validité.



- La phase de réalisation

La phase de réalisation consiste à réunir les deux branches, permettant de mener une conception applicative et enfin la livraison d'une solution adaptée aux besoins. La branche du milieu comporte :

- La conception préliminaire, qui représente une étape délicate, car elle intègre le modèle d'analyse dans l'architecture technique de manière à tracer la cartographie des composants du système à développer,
- La conception détaillée, qui étudie ensuite comment réaliser chaque composant ;
- L'étape de codage, qui produit ces composants et teste au fur et à mesure les unités de code réalisées,
- L'étape de recette, qui consiste enfin à valider les fonctions du système développé.

Représentation



Inter-State School Of Higher Education
PAUL BIYA TECHNOLOGICAL CENTRE OF EXCELLENCE
P.O Box 13 719 Yaounde (Cameroon) Tel. (237) 22 72 99 58
Web site: www.iaicameroun.com E-mail: iaicameroun@yahoo.fr

